

DeepPersona 코드 학습교재

모듈 01 — 큰 그림: 무엇을 · 왜 · 어떻게 (코드 ↔ 논문 지도)

Jin Sam Cho · cho@kaist.ac.kr

September 13, 2026

원저: Wang et al., *DeepPersona* (arXiv:2511.07338)

이 모듈의 목표와 순서

한눈에 DeepPersona를 **한 장의 지도**로 먼저 이해한다 — 다음 모듈부터 코드를 줄 단위로 판다.

이 모듈에서 얻는 것

- 이 프로젝트가 푸는 문제
- 2단계 파이프라인의 전체 흐름
- **코드 파일** ↔ **논문 단계** 대응표
- 핵심 수식 2개(무엇을 뜻하는지)

학습 로드맵(서브폴더)

- `01_overview` — 지금 여기
- `02_taxonomy` — 속성 트리(Stage 1)
- `03_based_data` — 시드/앵커
- `04_select_attributes` — 5:3:2
- `05_generate_profile` — 값 채우기
- `06_run_reproduce` — 실행/재현

▶ 목표: 사용자가 코드를 이해하고 그대로 재현할 수 있게 한다.

1. 왜 만들었나 — 문제의식
2. 핵심 아이디어 — 2단계 파이프라인
3. 코드 ↔ 논문 지도
4. 핵심 수식 2개
5. 정리 & 다음

1. 왜 만들었나 — 문제의식

기존 합성 페르소나는 **얕다**

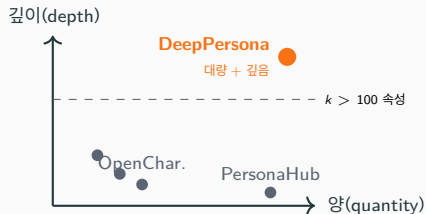
한눈에 기존 방법은 양(quantity)은 키웠지만 깊이(depth)는 못 키웠다.

합성 페르소나(synthetic persona)란?

- LLM에 "당신은 32세 게임개발자..."처럼 가상 인물 설정을 주입한 것
- 용도: 개인화 LLM 학습, 사회 시뮬레이션, 정렬(alignment) 연구

한계 (논문 §1)

- 보통 **30개 미만**의 수작업 속성뿐 $\Rightarrow$ 납작함
- 순진하게 LLM으로 늘리면 판박이(stereotype) · 낙관 편향



▶ 논문 Fig. 1을 개념만 다시 그림.

목표: narrative-complete 페르소나

쉽게 말하면 "이야기까지 완결된" 페르소나 = 속성이 수백 개인데(깊이), 서로 안 모순되고(일관), 사람마다 다른(다양) 인물.

논문은 좋은 페르소나가 갖춰야 할 세 조건을 못 박는다(§3):

깊이 Depth

속성 수 $k > 10^2$, 그리고 서술문 $\text{Narr}(P)$ 가 그 속성을 정확히 요약.

다양성 Diversity

인구 전체에서 속성·값의 분포가 실제 인간과 비슷.

일관성 Consistency

만들어진 사실들이 논리적으로 서로 모순되지 않음.

한눈에 이 중 **깊이**가 진짜 병목 — DeepPersona의 핵심 기여는 "깊이를 대량으로" 확보한 것.

2. 핵심 아이디어 — 2단계 파이프라인

두 단계로 나눈다: 트리를 짓고 → 트리를 걷는다

한눈에 Stage 1은 **속성 트리**(taxonomy)를 한 번 만들고, Stage 2는 그 트리를 걸으며 **사람마다** 페르소나를 채운다.

Stage 1 — 트리 구축 (한 번, 오프라인)



Stage 2 — 페르소나 생성 (사람마다, 온라인)



▶ 논문 Fig. 2의 흐름을 코드 관점으로 재구성. 점선 = 만들어 둔 트리를 매 페르소나가 재사용.

Stage 2 안을 더 자세히 — 5단계

쉽게 말하면 시드 인물을 만들고 → 그 사람에 어울리는 빈 슬롯들을 트리에서 골라 → LLM이 슬롯마다 값을 써 넣는다.

1. **시드 생성**: 나이·직업·가치관·인생이야기·취미를 먼저 정한다 (`based_data.py`)
2. **시드 임베딩**: 시드 요약문을 `text-embedding-ada-002` 로 벡터화
3. **유사도 계산**: 시드 벡터 vs 2,297개 슬롯 벡터의 코사인 유사도
4. **5:3:2 선택**: 가까움:중간:먼 = 5:3:2 로 슬롯 200개 추림 (`select_attributes.py`)
5. **값 채우기**: 고른 슬롯을 LLM에 주고 사람 맞춤 값을 생성 (`generate_profile.py`)

한눈에 매핑 방향이 직관과 반대다. 값→슬롯 분류가 아니라, 시드 인물→어울리는 슬롯 발견이다.

▶ 비유: 도서관 책장 라벨 2,297개 중 손님 관심사에 맞는 라벨을 고르고, 그 책장의 책은 작가(LLM)가 새로 쓴다.

논문 원본 개요 그림 (Figure 2)

쉽게 말하면 내 코드지향 도식과 함께 논문의 공식 개요 그림도 본다 — 세 패널(대화→트리 / 앵커 / 프로파일).

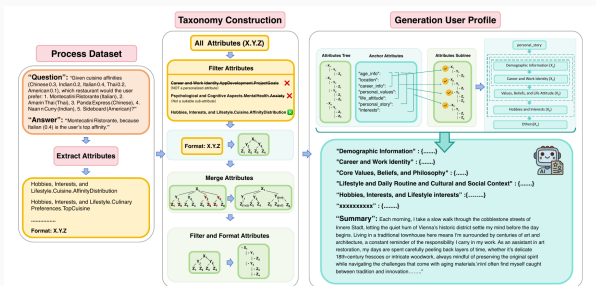


Fig. 2 (Wang et al.). 좌=Process Dataset · 중=Taxonomy Construction(Stage 1) · 우=Generation User Profile(Stage 2).

▶ 좌·중 패널 = 모듈 02(트리 구축), 우 패널 = 모듈 03-05(시드→선택→채우기).

3. 코드 ↔ 논문 지도

어느 파일이 논문의 어느 단계인가 (재현의 뼈대)

한눈에 재현하려면 "이 파일 = 논문 이 단계"를 외우면 된다. 아래가 그 대응표.

코드 (파일/함수)	논문 단계	하는 일
process_attributes/ data/large_attributes.json based_data.py select_attributes.py data/...embeddings.pkl generate_profile.py	Stage 1 §3.1 트리 산출물 §3.2 앵커 코어 §3.2 다양화 속도 캐시 §3.2 채우기	대화→속성 추출·병합·검증 2,297개 leaf 슬롯(경로) 나이·직업·값·인생이야기 시드 임베딩+코사인+5:3:2 선택 슬롯 벡터 lookup 테이블 슬롯마다 LLM 값 생성+Summary

▶ Stage 1은 한 번만 돌려 트리를 만들고 커밋해 둬. 평소 생성은 Stage 2(아래 4줄)만 반복.

5:3:2 — 코드에서 그대로 확인 (미리보기)

쉽게 말하면 고른 슬롯을 유사도 순으로 줄 세운 뒤, 위쪽(가까움) 5, 가운데(중간) 3, 아래쪽(먼) 2 비율로 뽑는다. 관련성과 의외성을 동시에.

```
# select_attributes.py (§5:3:2 발췌)
total_ratio = 5 + 3 + 2
near_count = int(target * 5 / total_ratio)
mid_count = int(target * 3 / total_ratio)
far_count = int(target * 2 / total_ratio)
```

- `near` = 유사도 상위 1/3 구간에서 표집(관련성)
- `mid` = 가운데 1/3(다양성)
- `far` = 하위 1/3(의외성)



▶ 4번 모듈에서 이 선택 로직을 줄 단위로 해부한다.

4. 핵심 수식 2개

수식 1 — 페르소나는 속성-값 쌍의 집합

쉽게 말하면 한 사람 = "(속성, 값)" 짝들의 모음. 예: (나이, 32), (직업, 게임개발자), (취미, 등산).

$$P = \{ \langle a_i, v_i \rangle \mid a_i \in \mathcal{A}, v_i \in \mathcal{V}_{a_i}, i = 1, \dots, k \}$$

기호 정의 (하나도 빠짐없이)

- P : 한 명의 페르소나(persona). 우리가 만들려는 결과물.
- a_i : i 번째 속성(attribute). 예: "나이", "직업". `large_attributes.json` 의 한 경로.
- $\mathcal{A}$: 가능한 모든 속성의 우주(트리 전체). $|\mathcal{A}| \approx 2,297$ leaf.
- v_i : 그 속성의 값(value). 예: 32, "게임개발자". LLM이 생성.
- $\mathcal{V}_{a_i}$: 속성 a_i 가 가질 수 있는 값 공간(범주/자유서술/리스트).
- k : 속성 개수(=깊이). DeepPersona는 $k > 100$ 을 목표(코드 기본 200).

▶ 논문 식 (1). $\mathcal{A}$ =슬롯 우주는 Stage 1이, v_i =값은 Stage 2 LLM이 만든다.

수식 2 — 한 슬롯씩 조건부로 채운다

쉽게 말하면 앞서 채운 내용($P_{<i}$)을 보고 다음 슬롯을 고르고(selector), 맥락에 맞는 값을 쓴다(generator).

$$P \sim \mathcal{F}_{\theta, T}(\cdot \mid S, k) = \prod_{i=1}^k \underbrace{\Pr(a_i \mid S, P_{<i}, T)}_{\text{selector (슬롯 고르기)}} \cdot \underbrace{\Pr_{\theta}(v_i \mid a_i, S, P_{<i})}_{\text{generator (값 쓰기)}}$$

기호 정의

- S : 앵커. 사용자가 준 시드(나이=35, 직업=간호사 ...). `based_data.py` 산출.
- T : 속성 트리. selector에게 가능한 슬롯·균형을 알려 줌. θ : LLM 파라미터(값 생성기).
- $P_{<i}$: 지금까지 채운 부분 페르소나(맥락). $\mathcal{F}_{\theta, T}$: 합성 함수 $(S, k) \mapsto P$.

한눈에 두 확률의 곱 = 슬롯 고르기 $\times$ 값 쓰기를 k 번 반복(식 2) = 코드의 Stage 2 루프.

5. 정리 & 다음

꼭 기억할 것

- 문제 = 페르소나의 **깊이**
- 해법 = 트리 짓고 → 트리 걷기 (2 stage)
- Stage 2 = 시드→5:3:2 슬롯 선택→LLM 값 채우기
- 매핑은 **인물→슬롯** (값→슬롯 아님)
- **pk1** 은 정확도가 아니라 속도용 캐시

정량 성과 (논문 Table 1)

지표	PH	OC	Ours
평균 속성 수	3.98	38.5	50.9
고유성	2.50	2.86	4.12
실행가능성	3.60	4.78	5.00

PH=PersonaHub, OC=OpenCharacter. 다양성 +32%, 고유성 +44%.

한눈에 다음 모듈 02_taxonomy : 이 "트리"가 파일 안에서 어떤 모양인지 연다.

모듈 01 끝

다음: 02_taxonomy — 속성 트리(Stage 1)